

Royal Enfield Tripper BLE Protocol

Reverse Engineering Notes (Work In Progress)

Version: 0.1

Status: Partial protocol reconstruction from APK reverse engineering.

1. Overview

The Royal Enfield Tripper module communicates over Bluetooth Low Energy (BLE). The Android application contains a `TripperBleManager` and `TripperProtocol` class responsible for:

- Pairing
- Authentication
- Sending navigation instructions
- Compass mode
- Time synchronization
- Firmware querying
- Receiving acknowledgements and status notifications

The firmware update protocol has intentionally been excluded from this document.

2. Important Classes

BLE Layer

TripperBleManager

Responsibilities:

- BLE connection management
 - Queueing packets
 - Writing packets
 - Receiving notifications
 - Handling reconnect logic
-

Protocol Layer

```
TripplerProtocol
```

Responsibilities:

- Packet construction
- Packet parsing
- CRC calculation
- Building navigation commands
- Building compass commands
- Building time commands
- Building PIN commands

3. Known Outgoing Commands

The following packet builders exist inside the APK.

Navigation

```
buildNavPacket(...)  
sendNav(...)
```

Parameters:

```
sendNav(  
    byte screen,  
    int dist,  
    byte maneuver,  
    byte heading,  
    byte speedFlags,  
    byte roadType,  
    byte eta  
)
```

Packet layout still under investigation.

Compass

```
buildCompassPacket(...)  
sendCompass(byte direction)
```

Used by:

```
CompassManager
```

Likely sends one of eight cardinal directions.

Possible values:

```
N  
NE  
E  
SE  
S  
SW  
W  
NW
```

PIN Authentication

```
buildPinPacket(String code)  
sendPin(String code)
```

Used during pairing and authorization.

Time Synchronization

```
buildSetTimePacket(...)  
buildSetTimeNowPacket(...)  
sendSetTime(...)  
sendSetTimeNow(...)
```

Synchronizes the bike clock.

Ping Firmware

```
sendPingFirmware()
```

Packet:

```
PING FW (0x03)
```

Ping Waypoint

```
sendPingWaypoint()
```

Packet:

```
PING WP (0x30)
```

Show PIN Screen

```
sendShowPin()
```

Packet label:

```
SHOW PIN (0x21 01)
```

Close / Resume

```
sendClose()
```

Packet label:

```
CLOSE (0x21 00)
```

Stop Navigation

```
sendStopNav()
```

Navigation Idle

```
sendNavIdle()
```

Raw Packet Injection

```
sendRawPacket(String hex)  
sendRawBytes(...)
```

Useful for protocol testing.

4. Send Queue Architecture

All packets are eventually passed through:

```
enqueuePacket(byte[] data, String label)
```

The manager maintains a transmission queue before writing packets to BLE.

5. Notification Handling

Notifications arrive through:

```
onCharacteristicChanged()
```

which calls:

```
handleNotification(byte[])
```

The notification is then parsed by:

```
TripperProtocol.parseResponse(...)
```

6. Known Response Labels

The parser currently recognizes the following labels.

AUTH

Examples:

```
AUTH ✓  
AUTH ✗
```

Behavior:

```
✓ -> onPinAccepted()  
✗ -> onPinRejected()
```

OS_VERSION

Example:

```
Firmware: x.x
```

Stored in:

```
lastTripperFirmware
```

SERIAL

Returns device serial information.

SESSION

Purpose still unknown.

TIME_ACK

Acknowledgement of time synchronization.

NAV_ACK

Acknowledgement of navigation packet.

NACK

Generic command failure.

Logged as:

ERROR

7. Notification Log Format

Notifications are logged as:

► [LABEL] Description

Examples:

- ▶ [AUTH] ✓
- ▶ [OS_VERSION] Firmware: 1.3
- ▶ [NAV_ACK] Accepted

8. Packet Format

The APK reveals that every packet is:

20 bytes

Layout:

Bytes 0-17 Payload
Bytes 18-19 CRC16

Diagram:

```
+-----+-----+
| Payload (18 bytes) | CRC16  |
+-----+-----+
|      0..17        | 18..19  |
+-----+-----+
```

9. CRC Algorithm

The protocol uses:

CRC-16/CCITT-FALSE

Parameters:

Polynomial : 0x1021
Init : 0xFFFF
RefIn : false


```
RefOut      : false
XorOut      : 0x0000
```

CRC bytes are stored in big-endian order:

```
packet[18] = crc >> 8
packet[19] = crc & 0xFF
```

10. Python Implementation

```
def crc16_ccitt_false(data):
    crc = 0xFFFF

    for b in data:
        crc ^= (b << 8)

        for _ in range(8):
            if crc & 0x8000:
                crc = ((crc << 1) ^ 0x1021) & 0xFFFF
            else:
                crc = (crc << 1) & 0xFFFF

    return crc
```

Appending CRC:

```
def append_crc(packet):
    crc = crc16_ccitt_false(packet[:18])

    packet[18] = (crc >> 8) & 0xFF
    packet[19] = crc & 0xFF

    return packet
```

11. Reconstructed BLE Stack

```
Android App
```

```
↓
```

```
TripperBleManager
  ↓
enqueuePacket()
  ↓
BLE Write Characteristic
  ↓
Tripper Device
  ↓
Notification
  ↓
handleNotification()
  ↓
parseResponse()
```

12. What Is Fully Understood

- ✓ BLE connection flow
- ✓ Notification handling
- ✓ Response parsing labels
- ✓ Packet size
- ✓ CRC algorithm
- ✓ Packet queueing
- ✓ Authentication callbacks
- ✓ Firmware version extraction

13. Remaining Unknowns

The exact payload structures still need extraction for:

```
buildNavPacket()
buildCompassPacket()
```

```
buildPinPacket()  
buildSetTimePacket()
```

Once these are decoded, an independent SDK can be written without using the original Royal Enfield application.

14. Future Goals

- Document all command opcodes.
- Decode navigation payload format.
- Decode compass payload format.
- Decode PIN payload format.
- Reimplement protocol in Python.
- Create an open-source Tripper SDK.
- Create a BLE emulator and testing suite.
- Build custom applications for Tripper displays.